

Classification of DeFi Services using Graph-Based Neural Networks

Interdisciplinary Project Report

Maximilian Kleinegger
Mat. Nr.: 12041500
TU Wien
`maximilian.kleinegger@student.tuwien.ac.at`

Supervisor: Dr. Bernhard Haslhofer *Co-Supervisor:* Prof. Dr. Allan Hanbury

Domain-specific Lecture: *TE250 From Idea to Enterprise*¹ (completed on 27.01.2026)

1 Abstract

Decentralized Finance (DeFi), based on Distributed Ledger Technologies (DLT), enables a peer-to-peer financial network without the need for a central coordinator, unlike traditional financial services. Furthermore, it is intended to be secure and preserve privacy, making it appealing to individuals. However, as shown in prior work, it is possible to classify entities within a blockchain based on their transactions using relatively simple data mining techniques and basic machine learning (ML) methods, such as *Logistic Regression* or *Decision Trees*. Given the inherent graph-based structure of blockchain data, Graph Neural Networks (GNNs) appear to be a promising approach. Therefore, this project develops a feature engineering method to transform the data into a suitable format for a GNN, allowing to beat state-of-the-art methods by 13 percentage points for a fair comparable setup, while being efficient with limited data and keeping this advantage with increasing amounts of data. All resources are available at <https://github.com/mkleinegger/defi-GNN>.

2 Motivation

The motivation for this project arises from the promise of Distributed Ledger Technologies (DLTs) to provide secure and privacy-preserving systems, making them highly appealing to users [5, 6]. However, prior work [5] shows that blockchain entities can be classified using relatively simple techniques, raising concerns about privacy. Given that blockchain data naturally forms a graph, where entities are nodes connected by transactions, and considering recent advances in deep learning, especially Graph Neural Networks (GNNs), this project explores how effectively these models can be applied to such tasks [9, 10]. This is especially relevant as the DeFi user base continues to grow². Therefore, this project evaluates the effectiveness and feasibility of employing more advanced machine learning techniques, potentially providing insights into the future direction of the field.

To gain insights and obtain meaningful results, the following research questions (RQs) will be explored:

RQ1: To what extent can GNNs be effectively used for entity classification in blockchain networks, and how do they compare to state-of-the-art traditional supervised machine learning methods in terms of accuracy, recall, precision and F1-score?

¹<https://tec.illinois.edu/academics/courses/te250-120261>

²<https://www.statista.com/statistics/1297745/defi-user-number/> Accessed on 08.02.2025

RQ2: How well do GNNs scale with increasing amounts of input data when classifying blockchain entities, and what are the computational trade-offs compared to traditional ML methods?

3 Methodology

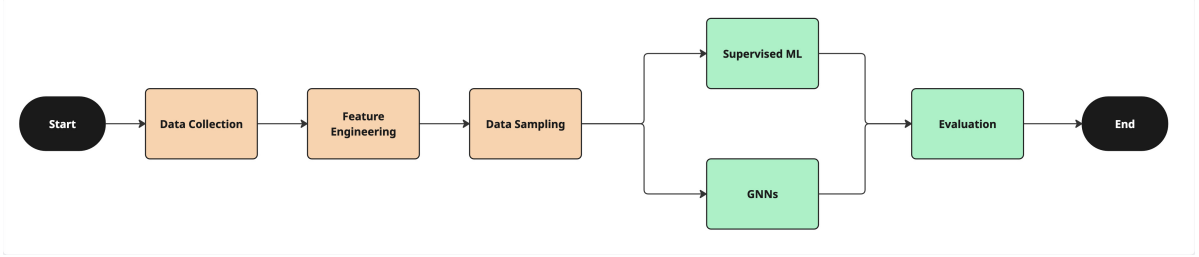


Figure 1: Overview of the experimental Pipeline.

This section will present the methodology that was applied to execute this project. We followed the CRISP-DM desing principles, as seen in Figure 1. We start by understanding our domain and our problem that we want to follow, as indicated by the start-block. We continue, by collecting and understanding the data, as indicated by the orange blocks. Lastly, we model the data either by using supervised ML or GNNs and afterward evaluate it, as indicated by the green blocks. Therefore, the following subsections will explain the individual parts in more detail.

3.1 Data

We start by explaining the most important part of the problem: **the data**. We describe how the data was collected, how it was processed, and which features were extracted.

3.1.1 Data Collection

Since our goal is to classify DeFi services using different ML techniques, we require transactional data from a distributed ledger technology (DLT). In this context, DeFi services can be understood as actors within a DLT that provide specific functionalities, such as exchanges, yield generation, or staking.

We were provided with Ethereum traces from the *Digital Currency Ecosystems research group* at the Complexity Science Hub. The dataset spans blocks 20,600,000 to 20,899,999 and contains 47,220,325 transactions across 10,120,062 unique addresses. Although this represents only a small portion of the full Ethereum ledger, it is sufficient for our experiments given the available computational resources.

In addition, we require labels for the addresses in the dataset. Manually labeling all addresses is not feasible within our computational and budget constraints. Therefore, we use the *DeFiLlama.com* API to retrieve metadata on Ethereum-based services, including their category and associated addresses. This allows us to construct a mapping between addresses and labels. All remaining addresses are assigned an *unknown* label.

3.1.2 Data Processing Pipeline

The raw data was initially stored across multiple files, which we combined into a single parquet file. This file was processed using Polars with an Apache Spark backend. The overall pipeline is illustrated in Figure 2.

First, all traces are merged into a unified dataset. Next, we enrich each transaction with its corresponding block timestamp. We then aggregate the data at the address level by grouping

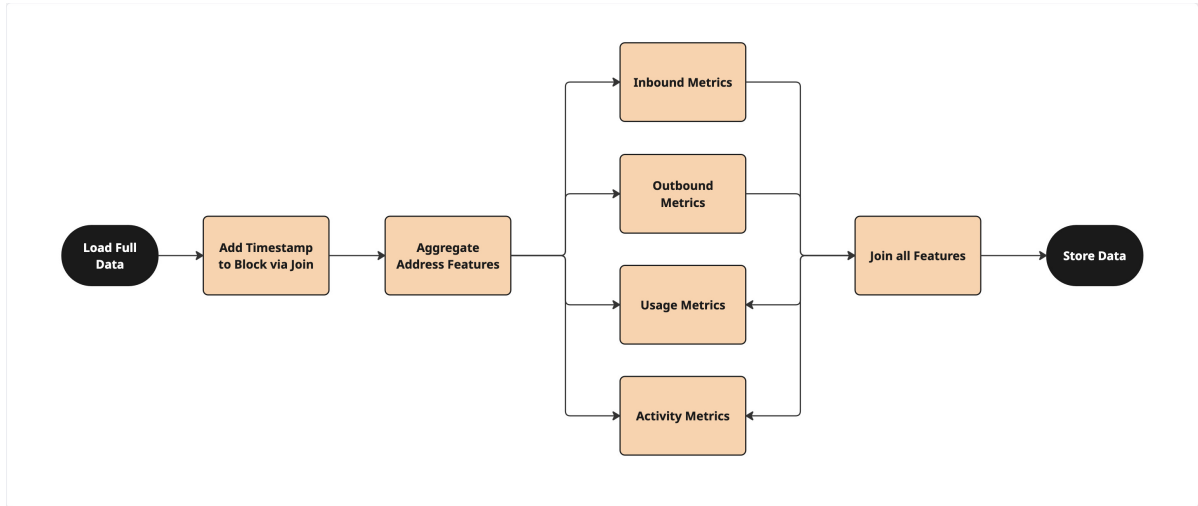


Figure 2: Data Processing Pipeline

transactions and computing relevant metrics. Finally, all computed features are joined and stored in a single dataset.

3.1.3 Feature Engineering

Based on the processed data, we compute a set of features inspired by prior work [5, 6]. These features can be grouped into the following categories:

- **Inbound Metrics:** Capture incoming transaction behavior, including the number of unique senders, transaction counts, received value, and temporal activity patterns.
- **Outbound Metrics:** Describe outgoing transaction behavior, including the number of unique recipients, transaction counts, transferred value, and gas usage.
- **Usage Metrics:** Reflect interactions with smart contracts, measured through the diversity of invoked methods (e.g., distinct function signatures).
- **Activity Metrics:** Characterize overall activity over time, including transaction frequency, active duration, and normalized activity rates.

Most features are derived from prior work. However, the *distinct function signatures* feature is our own contribution. Additionally, we compute the PageRank of the transaction graph as an extra feature. While computationally expensive, this was feasible due to the limited dataset size. Using lazy loading, all features can be computed efficiently at the address level and stored in a single CSV file. The dataset is available on our github repository.

3.1.4 Limitations

Using this dataset, we identified two key limitations: 1) Only 1,412 addresses have labels, and 2) the label distribution is highly imbalanced, as shown in Figure 3. These issues lead to poor performance in initial experiments and require careful handling.

3.1.5 Data Selection Strategy

To address these limitations, we explored three strategies:

1. **Including unknown labels:** Enrich the dataset with unlabeled addresses.

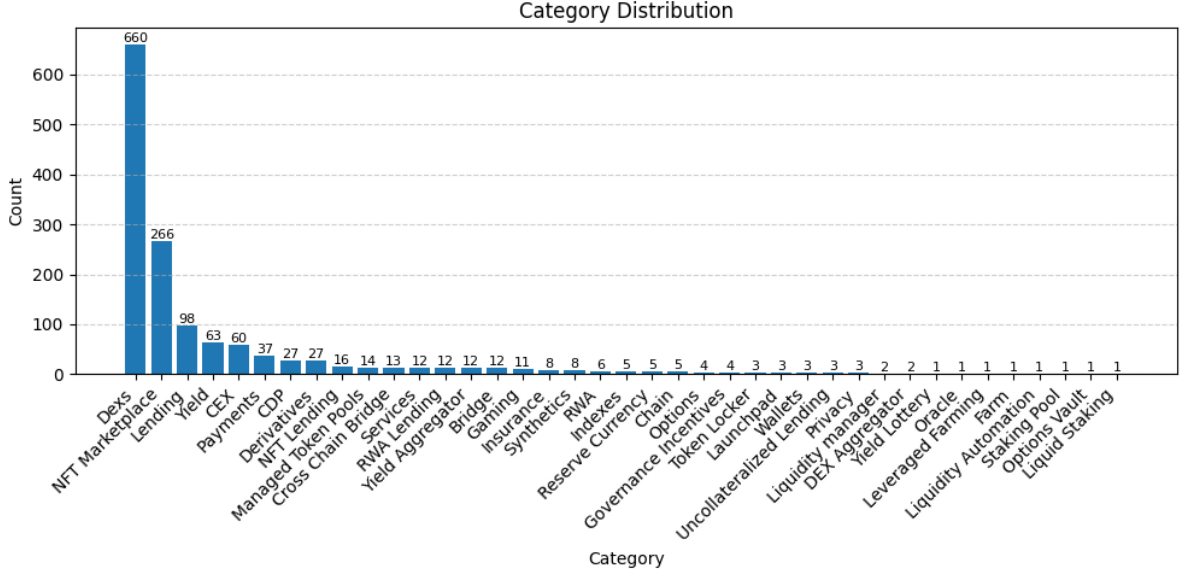


Figure 3: Visualization of the class imbalance within our dataset.

2. **Top-K filtering:** Restrict the dataset to the most frequent classes.
3. **Sampling:** Resample data points to balance the distribution.

Each approach has drawbacks. Including unknown labels introduces extreme class imbalance, leading to trivial majority-class predictions. Top-K filtering simplifies the task, while sampling introduces duplicated data and risks overfitting.

We therefore adopt a combined approach. First, we add approximately 800 unknown-labeled addresses. Then, we select the Top-10 most frequent classes, each containing at least 16 labeled addresses. This results in a dataset that remains challenging while being computationally feasible. This step is illustrated in the *Data Sampling* stage of Figure 1.

3.1.6 Final Data Configuration

Based on the chosen setup, we define the following data-related hyperparameters:

Model / Component	Hyperparameter	Value
Data	Random state	12041500
Data	Top categories	10
Data	Use PageRank feature	True
Data	Sample unknown addresses	True
Data	Number of unknown addresses	800

Table 1: Hyperparameter configuration of the supervised baseline models.

3.2 Machine Learning (ML)

Since our goal is to compare Graph Neural Networks (GNNs) with baseline approaches, we use supervised machine learning models suitable for classification tasks. We choose Decision Trees, Random Forests, Logistic Regression, and Multi-layer Perceptrons (MLPs), as these still perform well with proper data processing, and their results are comparable to prior work. We used the scikit-learn [7] implementations. Therefore, we use 5-fold cross-validation for training and evaluation. The hyperparameter configuration of the models is listed in Table 2.

Model / Component	Hyperparameter	Value
General	Random state	12041500
Logistic Regression	max_iter	100000
Logistic Regression	random_state	12041500
Random Forest	n_estimators	100
Random Forest	random_state	12041500
MLP Classifier	hidden_layer_sizes	(1024)
MLP Classifier	activation	ReLU
MLP Classifier	solver	Adam
MLP Classifier	max_iter	25000
MLP Classifier	random_state	12041500
Decision Tree	random_state	12041500

Table 2: Hyperparameter configuration of the supervised baseline models.

3.2.1 Graph Neural Networks (GNNs)

GNNs [3] are a specialized class of deep learning models designed to operate directly on graph-structured data. In our setting, we represent the DLT as a graph, where addresses correspond to nodes and transactions correspond to edges.

Following this approach, we implement a GraphSAGE-based neural network [4] consisting of three graph convolutional layers. The first two layers map input node features to a hidden representation, while the final layer produces class logits for node classification. Each intermediate layer is followed by batch normalization, a ReLU activation function, and dropout for regularization, which stabilizes training and helps prevent overfitting.

We chose GraphSAGE as it explicitly aggregates information from neighboring nodes at each layer, allowing the model to capture both local graph structure and node-level feature information.

Implementation We use the PyTorch Geometric (PyG) [1, 2] library, which is built on top of PyTorch. It provides efficient abstractions for graph data, where a graph is represented by a set of nodes, edges, and a feature matrix. Additionally, we maintain a mapping from node indices to labels for classification.

Neighbor Sampling Since the full transaction graph is very large, training on the complete graph is not feasible. Therefore, we use a neighborhood sampling approach via the `NeighborLoader` [4]. Instead of processing the full graph, we sample a subgraph for a given batch of target nodes.

Concretely, we first select a set of seed nodes (e.g., training nodes), and then iteratively sample a fixed number of neighbors per hop. In our case, we use a two-hop sampling strategy with (150, 20) neighbors per seed node. This means that for each seed node, up to 150 first-hop neighbors are sampled, and for each of those, up to 20 second-hop neighbors are included. This significantly reduces memory consumption while still preserving sufficient local structure for learning. The loader dynamically constructs mini-batches based on these sampled subgraphs.

Training Due to the limited amount of labeled data and high variability in the dataset, training for a fixed number of epochs did not yield stable results. Instead, we adopt a step-based training procedure, inspired by [8], where each step corresponds to one forward and backward pass on a sampled batch.

We observed that larger batch sizes lead to better performance, as they allow the model to capture more variability within each update step.

In general, we use the same data and data processing pipeline as for the baseline. More specifically, we use a 70/10/20 split for training, validation, and testing, and repeat each experiment across five random seeds. To ensure a fair comparison, we use the same set of addresses as in the baseline. Additionally, we include the same aggregated features as node features, with the exception of removing all graph-related features, i.e., in- and out-degree and PageRank, as relying solely on the graph structure is not sufficient.

Hyperparameters All training parameters are summarized in Table 3. These were selected via ablation experiments by choosing the best performing configuration.

Component	Hyperparameter	Value
General	Random seed	12041500 (+ 1,2,3,4)
Model	Hidden dimension	4096
Model	Number of layers	3
Model	Dropout	0.15
Model	Aggregation	Mean
Training	Learning rate	0.003
Training	Weight decay	1×10^{-6}
Training	Label smoothing	0.0
Training	Batch size	128
Training	Neighbors per layer	(150, 20)
Training	Max steps	5000
Training	Evaluation interval	200
Training	Max runtime	600s

Table 3: Hyperparameter configuration of the GraphSAGE model.

4 Results

This section presents the final results of the project. We begin by identifying the best-performing baseline model, then compare it to our trained GNN, and conclude by evaluating the scalability of GNNs.

4.1 Baselines

Model	Accuracy	Precision	Recall	F1
Decision Tree	0.6514	0.5172	0.5269	0.5149
Logistic Regression	0.3306	0.2597	0.2367	0.2103
MLP Classifier	0.2764	0.2132	0.1969	0.1632
Random Forest	0.7014	0.5791	0.5638	0.5577

Table 4: Performance of supervised models on the test set with Top-10 classes.

We start by presenting the results for the baselines in Table 4. As shown, Random Forest outperforms every other method. Furthermore, we observe that tree-based methods clearly outperform the remaining approaches. Furthermore, we want to highlight that those

4.2 Baseline vs GNNs

Moving to the comparison between our best baseline (Random Forest) and the GNN, the results can be seen in Table 5. For the Top-10 labels, we improve accuracy by approximately

Model	Top-K	Accuracy	Precision	Recall	F1
RandomForest	10	0.7014	0.5791	0.5638	0.5577
GNN	10	0.8326	0.6167	0.5265	0.5586
RandomForest	42	0.4563	0.2999	0.2849	0.2775
GNN	42	0.7624	0.3219	0.2359	0.2527

Table 5: Comparison of Random Forest and GNN performance across top-10 and all categories

13 percentage points, while slightly improving precision by 4 percentage points and decreasing recall by 4 percentage points. The F1-score remains roughly unchanged. Therefore, we are able to improve over our baseline and achieve our main goal.

Furthermore, we evaluate the models on the full dataset. As shown, the Random Forest performs rather poorly, achieving only an accuracy of $\approx 45\%$. In contrast, the GNN reaches $\approx 76\%$, making it considerably better suited for this task and more generally applicable. However, precision and recall, and consequently the F1-score, remain relatively low.

Therefore, the GNN clearly outperforms our best baseline. This is particularly notable, as a simple MLP classifier trained on the same features performed worst among all baselines. This further highlights the advantage of GNNs when working with graph-structured data.

4.3 Scalability Evaluation

One major concern when implementing GNNs is how well they scale with increasing data size. This can largely be addressed by the use of the **NeighborLoader**, which enables sampling of large graphs into smaller subgraphs that can be processed efficiently during training. By selecting a subset of seed nodes, we can reduce a graph with millions of nodes to mini-batches containing only a few thousand nodes, depending on the chosen neighborhood size.

Further hyperparameter tuning allows us to control how many neighbors are sampled per node and how many hops are considered, providing a trade-off between computational efficiency and the amount of structural information captured. A potential limitation arises in very dense graphs, where larger neighborhoods may be required to capture meaningful structure. However, this can be mitigated by reducing the batch size, keeping the overall computation manageable.

In our experiments, we used a GPU but required only around 1.5GB of VRAM and approximately five minutes of training time. This suggests that even CPU-based training is feasible without excessive memory requirements and can still be completed within a reasonable time frame.

While for our relatively small dataset the runtime of GNNs is higher compared to classical supervised methods (minutes vs. seconds), this trade-off changes at larger scales. For very large numbers of addresses, traditional methods can become computationally expensive, whereas GNNs with efficient neighbor sampling may offer better scalability and overall efficiency.

5 Conclusion

This report demonstrates the effective and efficient use of GNNs for entity classification of DeFi services. We first established a strong baseline using well-known feature engineering techniques, addressed key data limitations, and successfully implemented and scaled a GNN-based approach. Compared to the baseline, we achieved an improvement of 13 percentage points, increasing performance from 70% to 80% under limiting the number of labels, for fairness, and from 45% to 76% when considering all labels. Furthermore, we showed that this approach is computationally feasible by leveraging the **NeighborLoader**, which enables efficient subgraph sampling. This allows training to remain memory-efficient and practical on both GPU and CPU hardware.

References

- [1] FEY, M., AND LENSSEN, J. E. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds* (2019).
- [2] FEY, M., SUNIL, J., NITTA, A., PURI, R., SHAH, M., STOJANOVIĆ, B., BENDIAS, R., BARGHI, A., KOCIJAN, V., ZHANG, Z., HE, X., LENSSEN, J. E., AND LESKOVEC, J. Pyg 2.0: Scalable learning on real world graphs, 2025.
- [3] HAMILTON, W. L. Graph representation learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning* 14, 3, 1–159.
- [4] HAMILTON, W. L., YING, R., AND LESKOVEC, J. Inductive representation learning on large graphs. *CoRR abs/1706.02216* (2017).
- [5] JOURDAN, M., BLANDIN, S., WYNTER, L., AND DESHPANDE, P. Characterizing entities in the bitcoin blockchain. In *2018 IEEE International Conference on Data Mining Workshops (ICDMW)* (2018), pp. 55–62.
- [6] NIEDERMAYER, T., SAGGESE, P., AND HASLHOFER, B. Detecting financial bots on the ethereum blockchain. In *Companion Proceedings of the ACM Web Conference 2024* (New York, NY, USA, 2024), WWW '24, Association for Computing Machinery, p. 1742–1751.
- [7] PEDREGOSA, F., VAROQUAUX, G., GRAMFORT, A., MICHEL, V., THIRION, B., GRISEL, O., BLONDEL, M., PRETTENHOFER, P., WEISS, R., DUBOURG, V., VANDERPLAS, J., PASSOS, A., COURNAPEAU, D., BRUCHER, M., PERROT, M., AND DUCHESNAY, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [8] VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, L., AND POLOSUKHIN, I. Attention is all you need. *CoRR abs/1706.03762* (2017).
- [9] WEBER, M., DOMENICONI, G., CHEN, J., WEIDELE, D. K. I., BELLEI, C., ROBINSON, T., AND LEISERSON, C. E. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *CoRR abs/1908.02591* (2019).
- [10] ZOLA, F., SEGUROLA-GIL, L., BRUSE, J. L., GALAR, M., AND ORDUNA-URRUTIA, R. Attacking bitcoin anonymity: generative adversarial networks for improving bitcoin entity classification. *Applied Intelligence* 52, 15 (2022), 17289–17314.

6 Appendix

6.1 Data Science Curriculum

For the report specification, we include the various techniques from the Data Science curriculum that were used in the project. The main methods applied are detailed below.

1. **Data collection:** To perform the necessary experiments, data from an actual DLT must be extracted. This process will draw upon knowledge from courses such as Data-Oriented Programming Paradigms³ (188.995) and Experiment Design⁴ (188.992).
2. **Feature Engineering:** To establish a state-of-the-art baseline for comparison and to construct a feasible input for Graph Neural Networks (GNNs), we will perform feature engineering. This process will draw upon knowledge from courses such as *Data-oriented Programming Paradigms* (188.995), where we learned data preprocessing techniques, and *Algorithmics*⁵ (186.814), which enhanced our understanding of graph problems.
3. **Graph Neural Networks (GNNs):** The GNN is the crucial component of this project. As a deep learning model, it leverages concepts covered in the *Machine Learning*⁶ (184.702) lecture.
4. **Experimental Design:** To evaluate the results and perform statistical comparisons, we will design the experiment based on methodologies taught in the *Experiment Design* (188.992) course, ensuring that the findings are reliable and reproducible.

6.2 Overview of Generative AI Tools Used

This section lists for full transparency all AI Tools used during this project:

1. **ChatGPT** was utilized to edit or improve the clarity and understanding of the existing and self-written text. We mostly used the prompt "Please check for spelling and understanding" followed by our own text to enhance its clarity and readability. If the responses were not satisfying, we modified or even discarded them. Therefore, the content of the text was self-written all the time and this tool was only used to improve the clarity and understanding of it.
2. **GitHub Copilot** was used during development for fast prototyping, which was then converted into the code used for all experiments.

6.3 Full Feature List

The detailed result of the feature engineering can be seen in Table 6.

³<https://tiss.tuwien.ac.at/course/educationDetails.xhtml?courseNr=188995>

⁴<https://tiss.tuwien.ac.at/course/educationDetails.xhtml?courseNr=188992>

⁵<https://tiss.tuwien.ac.at/course/educationDetails.xhtml?courseNr=186814>

⁶<https://tiss.tuwien.ac.at/course/educationDetails.xhtml?courseNr=184702>

Category	Feature
Inbound Metrics	in_degree
	tx_count_in
	eth_received
	tx_min_in
	tx_max_in
	tx_mean_in
	avg_inter_tx_block_gap
Outbound Metrics	out_degree
	tx_count_out
	eth_sent
	tx_min_out
	tx_max_out
	tx_mean_out
	gas_used
Usage Metrics	distinct_method_count
Activity Metrics	active_block_span
	tx_total
	active_days
	fTX
	pagerank
Label	category

Table 6: Feature Overview